ОТЧЕТ ПО РАЗРАБОТКЕ ПРОЕКТА

"Разработка и тестирование синтаксического анализатора HTML-страниц для автоматизированного извлечения структурированных данных"

---

1. Технологический стек разработки

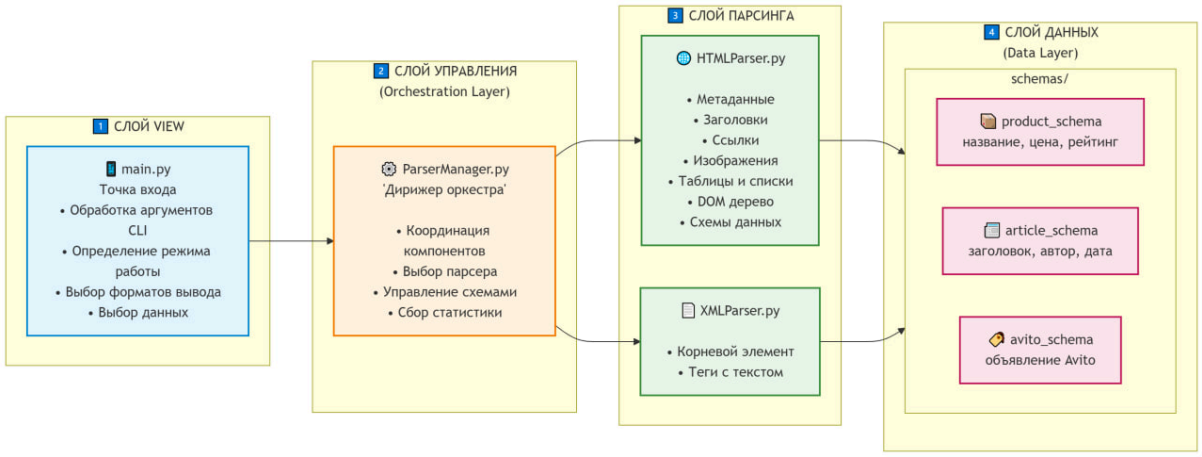
Компонент	Технология	Обоснование выбора
Язык программирования	Python 3.13	Высокоуровневый язык с богатой экосистемой для парсинга данных
Парсинг HTML/XML	BeautifulSoup 4 (bs4)	Гибкий и отказоустойчивый парсер с простым API
Парсер HTML	html.parser (встроенный)	Не требует дополнительных зависимостей, достаточная производительность
Парсер XML	lxml	Оптимальная скорость парсинга XML-документов

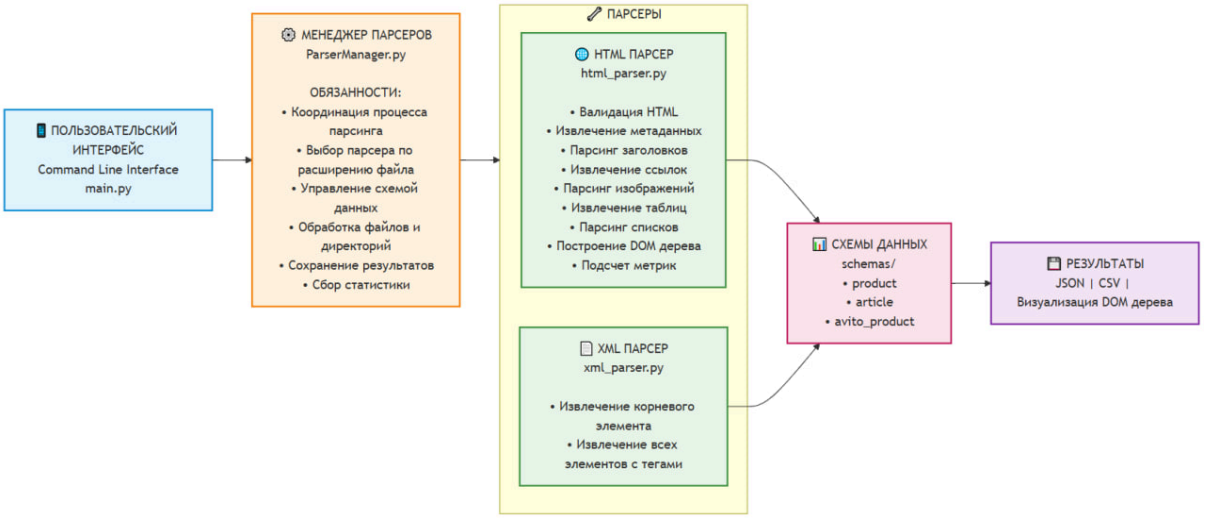
Формат данных	JSON / CSV	Универсальные форматы для обмена и анализа данных
Тестирование	pytest	Стандарт индустрии, простота интеграции

2. Архитектура приложения

2.1 Общая структура

```text
project/
├── main.py # Точка входа, CLI-интерфейс
├── src/
│   ├── parser_manager.py        # Оркестрация процессов парсинга
│   ├── html_parser.py           # Парсер HTML-документов
│   └── xml_parser.py            # Парсер XML-документов
├── schemas/
│   ├── schemas.py               # Централизованное управление схемами
```





3. Реализованная функциональность

3.1 Базовые возможности парсинга

Компонент	Функциональность	Статус
HTMLParser	Валидация HTML-документов	Реализовано
	Извлечение метаданных (title, description, keywords)	Реализовано
	Извлечение заголовков (h1-h6)	Реализовано
	Извлечение параграфов	Реализовано
	Извлечение ссылок с текстом	Реализовано
	Извлечение изображений (src, alt)	Реализовано
	Извлечение таблиц (заголовки + строки)	Реализовано

	Извлечение списков (маркированных и нумерованных)	Реализовано
	Построение DOM-дерева	Реализовано
	Визуализация DOM-дерева в консоли	Реализовано
	Подсчет метрик документа	Реализовано
	Очистка текста от лишних пробелов и переносов	Реализовано

Компонент	Функциональность	Статус
XMLParser	Извлечение корневого элемента	Реализовано
	Извлечение всех тегов с текстом	Реализовано

3.2 Система схем для структурированных данных

Реализована гибкая система извлечения структурированных данных на основе схем:

```python
# Пример схемы товара
product_schema = {
    "container": ("div", "product"),
    "fields": {
        "name": ("h2", None),
        "price": ("span", "price"),
        "rating": ("span", "rating")
    }

}
```

Схема	Назначение	Поля
product_schema	Товары интернет-магазина	name, price, rating
article_schema	Статьи и новости	title, author, date
avito_schema	Объявления Avito	title, price, condition

3.3 Менеджер парсеров (ParserManager)

Реализован класс-оркестратор, обеспечивающий:

Функция	Описание
Выбор парсера	Автоматическое определение типа файла по расширению (.html / .xml)
Управление схемами	Фильтрация схем по запросу пользователя
Обработка файлов	Чтение, парсинг, сбор статистики
Обработка директорий	Рекурсивный обход всех поддерживаемых файлов
Экспорт данных	Сохранение в JSON и CSV с созданием директорий
Статистика	Подсчет обработанных файлов, ошибок, найденных элементов

3.4 Интерфейс командной строки (CLI)

Реализован гибкий CLI с поддержкой различных режимов работы:

Параметр	Назначение	Пример
`path`	Путь к файлу или директории	`sample.html`
`--format`	Формат экспорта (json/csv)	`--format csv`
`--visualize`	Визуализация DOM дерева	`--visualize`
`--output-dir`	Директория для результатов	`--output-dir ./results`
`--schemas`	Выбор схем для извлечения	`--schemas products avito`
`--all-data`	Парсинг всех данных	`--all-data`
`--structured-only`	Только структурированные данные	`--structured-only`
`--common-only`	Только общие данные	`--common-only`
`--data-types`	Конкретные типы данных	`--data-types headings links`

3.5 Обработка реальных данных

Проведена адаптация для парсинга реальных веб-страниц на примере Avito:

4. Обеспечение тестируемости кода

При разработке уделено особое внимание возможности написания юнит-тестов:

Принцип	Реализация
Разделение ответственности	Каждый класс решает одну задачу (SRP)
Инъекция зависимостей	Парсеры передаются в менеджер через конструктор
Отсутствие глобального состояния	Все данные передаются явно
Чистые функции	Методы не имеют побочных эффектов
Обработка ошибок	Все исключения перехватываются и логируются

Результат: 141 юнит-тест с покрытием > 95% (по отчету QA-инженера).

5. Исправление ошибок по результатам тестирования

В ходе взаимодействия с QA-инженером были выявлены и исправлены следующие ошибки:

БАГ #1 - Несоответствие метрики links_count [В ПРОЦЕССЕ]

Проблема: Метод `get_document_metrics()` считал все теги `<a>`, включая те, у которых нет атрибута `href`, в то время как `extract_links()` возвращал только ссылки с `href`. Расхождение на реальной странице Avito составило 11 элементов.

Статус: Требуется доработка для приведения метрики в соответствие с реальными данными.

БАГ #2 - ParserManager.schemas пустой по умолчанию [ИСПРАВЛЕН]

Проблема: При рефакторинге был удален импорт схем, из-за чего `self.schemas` всегда был пустым словарем.

Решение: Реализована передача схем через конструктор `ParserManager(schemas=...)` и метод `set_schemas()`. Создан централизованный модуль `schemas/__init__.py` для управления схемами.

Статус: Исправлено, схемы корректно применяются при парсинге.

БАГ #3 - data_options не принимался HTMLParser.parse() [ИСПРАВЛЕН]

Проблема: Метод `HTMLParser.parse()` принимал только два аргумента, но `ParserManager.process_file()` передавал третий - `data_options`. Любой вызов завершался `TypeError`.

Решение: Обновлена сигнатура метода `parse()` - теперь принимает `data_options` с значением по умолчанию `None`. Добавлена логика фильтрации данных в соответствии с опциями.

Статус: Исправлено, все режимы фильтрации данных работают корректно.

---

6. Взаимодействие с QA-инженером

В процессе разработки было организовано тесное взаимодействие с QA-инженером:

Этап	Действия

Проектирование	Обсуждение тест-кейсов на этапе проектирования API
Разработка	Предоставление документации по методам и схемам
Тестирование	Консультации по неясным моментам в требованиях
Исправление ошибок	Оперативный фикс багов по результатам тестирования
Финальный отчет	Предоставление материалов для итогового отчета